

I Built a *10M Events/Day* Fraud Detection Pipeline. Here's What I Learned.

Six months at Western Union redesigning fraud detection from batch to streaming — the architecture, the hard tradeoffs, and the three things I'd do differently if I started today.

10M+

EVENTS PER DAY

<1s

END-TO-END LATENCY

3

STREAM TYPES UNIFIED

The System I Inherited

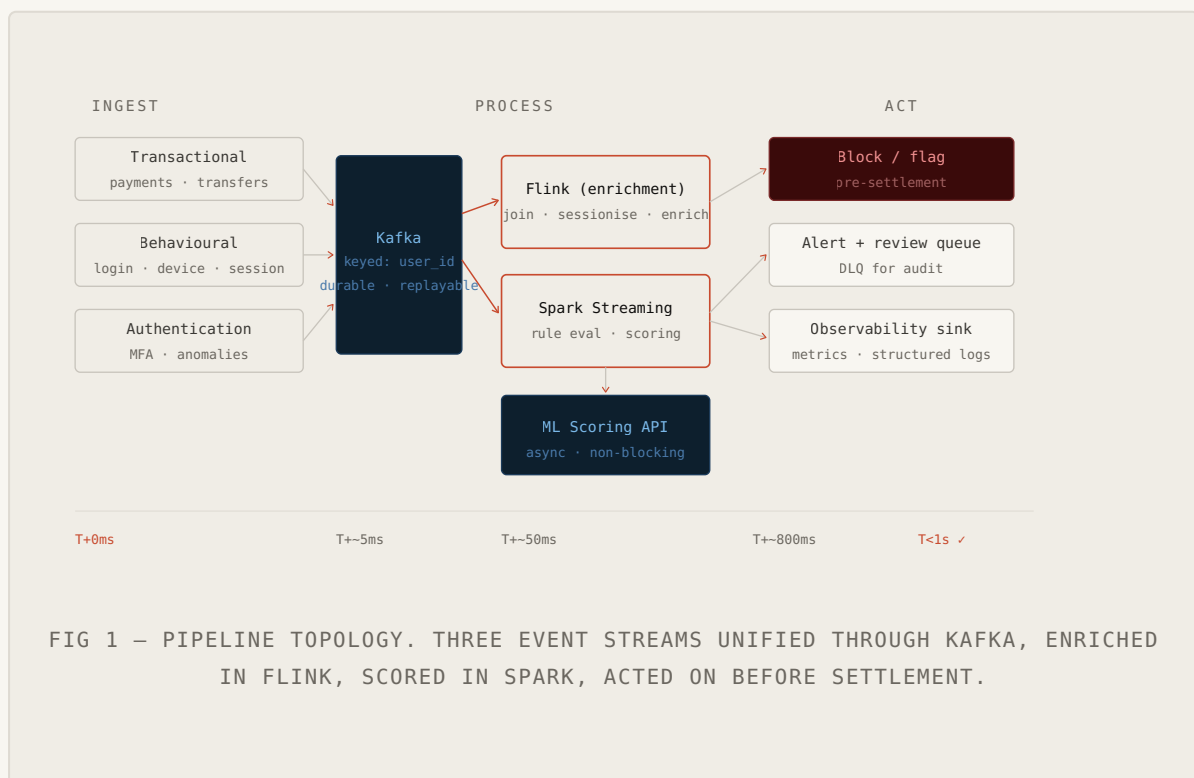
When I joined Western Union's fraud engineering team, the detection pipeline ran on periodic batch jobs. Risk scoring happened on a pull cycle — pull from the transaction database, run scoring, flag accounts. The cycle took minutes. At a company moving billions of dollars daily, that meant the fraud team was always working in the past.

The consequence was structural, not just operational: **we could identify fraud, but we couldn't prevent it.** By the time a pattern surfaced, the money had already moved. Every intervention was remediation. The system was forensic when it needed to be preventive.

My mandate was clear — rebuild this as a streaming pipeline that could act before settlement. What wasn't clear was how much the details would matter.

Why I Designed It the Way I Did

The core constraint was non-negotiable: fraud scoring had to complete before the transaction settled. That single requirement shaped every decision downstream. If you keep it in mind, most of the architecture becomes obvious. If you lose sight of it, you end up building something technically impressive that doesn't actually solve the problem.



WHY KAFKA AND NOT DIRECT CONSUMPTION

An earlier internal proposal had Flink consuming directly from upstream REST webhooks. I pushed back on this. Kafka gives you something REST fundamentally can't: **durable, replayable,**

partitioned ordering. When a downstream job crashes at 3am during a traffic spike — and it will — Kafka holds the events. The job restarts and resumes from its last checkpoint. With direct REST consumption, those events are gone. In a financial fraud context, that's not a data loss problem, it's a regulatory one.

I partitioned by `user_id` rather than random round-robin. This ensures all events for a given user land on the same Kafka partition, which means a single Flink task sees the complete context for that user — making stateful sessionisation possible without expensive cross-partition coordination.

WHY FLINK AND SPARK AS SEPARATE STAGES

Flink handles the stateful, latency-sensitive work: joining the three event streams within a time window, building per-user session context, and enriching events with historical features. This is where cross-stream intelligence lives. A payment event looks completely different when you also know the user just authenticated from a new device in a different country and their behavioural session is atypical.

Spark Streaming sits downstream for rule evaluation. I separated them deliberately. Flink is genuinely better at low-latency stateful joins. Spark is better at running complex analytical rule sets across enriched records in micro-batches. Using one engine for both means accepting its worst tradeoffs for half the job. At this throughput, that matters.

THE ML INTEGRATION DECISION I'M MOST PROUD OF

ML scoring is called asynchronously, as a side-channel from Spark. Making it synchronous would have added 200–400ms per event — straight through the latency budget. Instead, the ML score is

fetches async and merged with rule output before the final risk decision is emitted. If the ML service degrades or times out, we fall back to rule-only scoring. The pipeline never blocks on a model.

The one architectural constraint that drove every decision: scoring had to happen before settlement, not after. Lose sight of that and you've built something impressive that solves the wrong problem.

The Parts That Were Actually Hard

BACKPRESSURE AT SUSTAINED LOAD

At 10M events/day the pipeline sustains roughly 115 events per second, with significant spikes around pay cycles. Without backpressure control, a slow downstream — the ML scoring service, the action layer — backs up the enrichment layer, which backs up Kafka consumers, which eventually causes Flink task managers to OOM.

I implemented Flink's credit-based backpressure with dynamic parallelism — scaling task parallelism up and down based on consumer lag metrics. The critical tuning decision: **backpressure slows ingestion rather than drops events**. Dropping a transactional event in a fraud pipeline is a regulatory failure, not just a data quality one.

THE EXACTLY-ONCE DEBATE

We had a long internal debate about whether to use Flink's exactly-once processing mode via two-phase commit to Kafka. I chose at-least-once with idempotent consumers instead. Exactly-once at this throughput added around 15% latency overhead from coordination — too much given our budget. More importantly, our action layer was already designed to handle duplicates safely.

```
// Idempotent handler – safe under at-least-once delivery fun
handleFraudSignal(event: FraudSignal) { val key = "fraud:$
{event.txnId}:${event.signalType}" if (redis.setNX(key, event.score,
ttl = 24h)) { // First occurrence – act on it
  actionRouter.dispatch(event) } // Duplicate – already processed,
  discard silently }
```

Blocking a transaction twice is a no-op. Adding a duplicate alert to the review queue is filtered at read time. The simplicity of this tradeoff — give up exactly-once guarantees at the engine level, enforce idempotency at the application level — is something I'd apply again without hesitation.

GETTING RULES OUT OF THE DEPLOYMENT CYCLE

Early on, the fraud analysts were identifying new patterns faster than engineering could ship rule updates. A mule-account pattern detected Monday needed a rule live Tuesday, not in the next sprint. This isn't a process problem — it's an architecture problem.

I externalised rule definitions to a database, hot-loaded by the Spark rule engine on a configurable polling interval. Adding or changing a rule became a database write. The fraud team owned their detection logic directly. Time-to-detection for new patterns

went from days to under an hour. That change had more business impact than any single technical decision in the project.

What I'd Do Differently

-
- | | |
|---|--|
| 01 Build observability from day one, not week twelve. | We added the Prometheus and alerting layer in the final weeks. You cannot tune a streaming system you can't see — and you definitely can't debug a 3am incident without structured logs and latency histograms already in place. Production readiness is not a milestone. It's a prerequisite. |
|---|--|
-
- | | |
|--|--|
| 02 The Flink/Spark split is right architecturally, but costly operationally. | Two runtimes, two deployment configs, two sets of runbooks. If I were starting over today I'd evaluate whether Flink alone could absorb the rule evaluation layer. The cognitive overhead of operating two systems is a real and ongoing cost that doesn't show up in the initial design review. |
|--|--|
-
- | | |
|---|---|
| 03 The ML circuit breaker should have been a day-one requirement. | The fallback to rule-only scoring when the ML service degrades was added reactively after a staging incident. Any time you have a hard dependency on an external service in a latency-sensitive path, the degraded-mode behaviour needs to be specced before you write the happy-path code. |
|---|---|
-

04 **Schema registry from the start.** Kafka schemas evolved. Event shapes changed mid-project. We ended up with a compatibility shim that worked but nobody was proud of. Avro with a schema registry, enforced at the producer level from day one, would have saved two weeks of pain and prevented an entire class of production surprises.

What This System Taught Me

Rebuilding this pipeline reinforced something I've come to believe strongly: the right architecture for a problem is almost never the most technically interesting one. It's the one that most precisely fits the constraints — latency budget, failure tolerance, regulatory requirements, team operational capacity.

Every decision above was made against a specific constraint. Kafka over REST because of replay guarantees. At-least-once over exactly-once because of latency. Externalised rules because the feedback loop between detection and deployment was too slow. None of these are universally correct. They're correct for this system, at this scale, with these requirements.

That's the job. Not picking the best technology — picking the right technology for the problem in front of you, and being honest about the tradeoffs when you do.
